



MICROSERVICES COURSE

Session 23

Performance & Load Testing

k6 · Load Test Design · Watching Resilience Patterns Trigger for Real · Bottleneck Analysis



Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

DAY

Wednesday

TIME

3:00–5:30 PM

DURATION

2.5 Hours

PHASE

Phase 3 · Advanced & Enterprise

Where We Are



Everything built since Session 4 has been tested with a handful of curl commands. Today we put genuine concurrent load on the platform for the first time.

The First Time We See It Actually Run

- Every resilience pattern since Session 4 — exercised with a handful of manual curl calls
 - Every observability tool since Session 17 — tested with 15-request shell loops
 - Configuration written in application.yml is a HYPOTHESIS about how the system will behave
-

Today we test that hypothesis for real.

Prediction exercise: before any code runs, write down which resilience pattern YOU predict will show visible effects first, and at what request rate. We revisit this later.

Patterns Nobody Has Actually Seen Fire

Since Session 4 we configured Circuit Breakers, Bulkheads, TimeLimiters, and Retries — tested each with a handful of manual curl calls or a 15-request shell loop.

Honest question: has anyone actually seen ALL FOUR patterns fire, in the documented execution order, under REAL concurrent load — not a unit test mock, not 15 sequential curls, but genuine simultaneous traffic like a real Black Friday spike?

Prediction Exercise — write it on the board before any code runs:

“Which pattern do YOU predict will show visible effects first, at what request rate?”

We revisit these predictions after today's stress test demo.

Virtual User · Iteration · Threshold



Virtual User (VU)

One simulated client running the test script in a continuous loop for the test's duration.



Iteration

One full execution of the script's default function by one VU. A VU may complete many iterations.



Threshold

A pass/fail condition on a metric (e.g. $p(95) < 500\text{ms}$). Same 'fail the build' philosophy as S13's Quality Gate.

A VU Is an Abstraction, Not a Person Count



50 VUs hammering an endpoint as fast as possible generates VERY different load than 50 real users browsing casually.

- A VU is one simulated client executing the test script in a loop — a load-generation abstraction
- It is NOT a literal one-to-one mapping to a real human
- The mapping between VUs and real concurrent users is never one-to-one — interpret results with that in mind

Which Test Type Answers Which Question?

Does the test script even work, minimal load?

→ Smoke Test

Does the system behave correctly under NORMAL traffic?

→ Load Test

Where does the system actually BREAK?

→ Stress Test

Survive a sudden, short burst far above baseline?

→ Spike Test

Validating a specific resilience threshold?

→ Targeted Stress Test

Today's platform test: primarily a Load Test to establish a baseline, followed by a Stress Test (Lab 19) to find the breaking point.

smoke-test.js — The Simplest Possible k6 Script

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  vus: 1,
  duration: '10s',
  thresholds: {
    http_req_duration: ['p(95)<500'], // 95% of requests under 500ms
    http_req_failed: ['rate<0.01'], // less than 1% may fail
  },
};

export default function () {
  const res = http.get('http://localhost:8080/api/v1/products');
  check(res, { 'status is 200': (r) => r.status === 200 });
  sleep(1);
}
```

Reading k6's Summary Output

```
data_received.....: 1.2 MB  120 kB/s
data_sent.....: 45 kB  4.5 kB/s
http_req_duration.....: avg=42ms  p(95)=61ms  p(99)=88ms
  { expected_response:true }...: avg=42ms
http_req_failed.....: 0.00%  ✓ 500  ✗ 0
http_reqs.....: 500  50.02/s

✓ THRESHOLDS PASSED: p(95)<500, rate<0.01
```

order-load-test.js — Staged Ramp / Sustain / Ramp-Down

```
export const options = {
  stages: [
    { duration: '30s', target: 20 }, // ramp up to 20 VUs
    { duration: '2m', target: 20 }, // sustain 20 VUs — the LOAD TEST
    { duration: '30s', target: 0 }, // ramp down
  ],
  thresholds: {
    http_req_duration: ['p(95)<800', 'p(99)<2000'],
    http_req_failed: ['rate<0.05'],
  },
};
export default function () {
  const params = { headers: { Authorization: `Bearer ${__ENV.TEST_JWT}` } };
  const res = http.post('http://localhost:8080/api/orders', payload, params);
  check(res, { 'status 200/202': (r) => [200, 202].includes(r.status) });
}
```

The Test Needs a REAL JWT — No Bypass



```
TEST_JWT=<paste-a-valid-token> k6 run order-load-test.js
```

The test needs a real, valid JWT — either a cached Keycloak Client Credentials token (S20) or a pre-obtained customer token (S19).

Load testing operates against the FULL secured platform — not a bypassed test-only path. This gives an accurate picture of real production behavior.

stress-test.js — Deliberately Exceeding Thresholds

```
export const options = {
  stages: [
    { duration: '20s', target: 50 },
    { duration: '20s', target: 150 }, // well beyond S5's Bulkhead max-concurrent-calls: 10
    { duration: '20s', target: 0 },
  ],
  thresholds: {
    http_req_failed: ['rate<0.5'], // deliberately loose — we EXPECT failures
  },
};
// Same default function as order-load-test.js — only 'options' changes
k6 run stress-test.js

# Watch two terminals side by side while this runs:
#   curl localhost:8082/actuator/bulkheads
#   curl localhost:8082/actuator/circuitbreakers
```

Two Terminals — Watching Patterns Fire Live

```
/actuator/bulkheads
```

```
t=0s:  available-concurrent-calls: 10
```

```
t=8s:  available-concurrent-calls: 3
```

```
t=12s: available-concurrent-calls: 0
```

Bulkhead full — rejecting calls FIRST

```
/actuator/circuitbreakers
```

```
t=0s:  state: CLOSED
```

```
t=12s: state: CLOSED
```

```
t=17s: state: OPEN
```

A few seconds LATER — failure rate crosses threshold

Session 5's Execution Order — Confirmed Live



Bulkhead drops to 0 available calls BEFORE Circuit Breaker flips to OPEN.

This is exactly Session 5's documented execution order, happening for real:

Bulkhead → TimeLimiter → CircuitBreaker → Retry

Bulkhead is OUTERMOST — the first gate a request passes through, so it is the first pattern with an opportunity to reject a request when concurrency exceeds its limit.

Were Your Predictions Right?

Back on slide 4, you wrote a prediction: which pattern fires first, at what request rate?

- Compare the board's predictions against what the two terminals actually showed
- Bulkhead fired first — was this what the group expected, or a surprise?
- Configuration written months ago (Session 4-5) just proved itself correct — or revealed a gap

A hypothesis that survives contact with real load is worth more than a hypothesis nobody tested.

From “A Request Failed” to “THIS Span, Right Here”

k6 tells you WHAT failed

Aggregate: “12% of requests exceeded 800ms in this stage.”



Zipkin tells you WHERE

Precise: “THIS trace, THIS span — the Feign call to Inventory — took 780ms.”

Find one failed request from the k6 run → locate the corresponding trace in Zipkin → identify exactly which span shows the failure or longest duration.

Lab 19 — Find the Platform's Actual Bottleneck

1

Write a Stress Test

k6/checkout-stress-test.js targeting the full chain; exceed both Bulkhead AND Circuit Breaker thresholds

2

Document the Firing Order

Record with timestamps; compare against Session 5's documented theoretical order

3

Correlate to a Zipkin Trace

Find one failed request; identify exactly which span shows the failure

Duration: 15 min in-session + complete as homework · Builds on: S4-5 Resilience, S17 Observability, S20 Security

Lab 19 — Acceptance Criteria

- ✓ A k6 stress test script exists in k6/checkout-stress-test.js, targeting the full chain
- ✓ The test run demonstrably exceeds at least one Session 4-5 resilience threshold
- ✓ A written record documents the ACTUAL order patterns activated, with rough timestamps
- ✓ This actual order is compared against Session 5's documented theoretical order
- ✓ At least one failed request is correlated to a specific Zipkin trace and span

Commit: `session-23: add-k6-stress-test-and-bottleneck-findings`

What We Added Today

 Capability Added

Verified Resilience Under Real Load

The platform's resilience configuration is no longer a hypothesis — it has been observed firing, in the documented order, under genuine concurrent traffic.

 Services Updated

- New k6/ directory: smoke, load, stress tests
- No application code changes
- Baseline documented (req/s, P95/P99 latency)

Enterprise E-Commerce Platform — After Session 23

Config+Eureka


Gateway


Resilience
 VERIFIED

K8s Pipeline


Observ.+CQRS


Security+Mesh


Adv. Patterns


Load Tested
 NEW



Daily Quiz

8 questions · 10 minutes · Google Forms or Kahoot



Session 23 Complete

Resilience Verified Under Real Load

Next: Session 24 — Architecture Clinic #2 + Phase 3 Wrap-Up

Monday, 3:00–5:30 PM, Online